

A Selectable Precision Backend for `BHPtoolkit.jl`: Float64, BigFloat, and MultiFloat

BHPtoolkit MST module

June 30, 2026

Abstract

The Mano–Suzuki–Takasugi (MST) solver in `BHPtoolkit.jl` computes homogeneous solutions of the radial Teukolsky equation. Several downstream quantities—the branch-cut integral of the Green’s function and high-multipole energy fluxes in particular—require many more correct digits than IEEE double precision provides, while the bulk of the run time is exploratory and double precision is more than enough. We add a single, uniform *precision backend* switch that lets the user select `Float64`, `BigFloat`, or `MultiFloat` (the error-free-transform extended precision of `MultiFloats.jl`). Because the MST code is already generic over `R<:AbstractFloat`, the integration is additive: a compatibility shim supplies the handful of transcendental and special functions `MultiFloats.jl` does not provide, and a dispatch helper converts inputs to the requested working type. Across a grid of ~ 200 Kerr/Schwarzschild modes the `MultiFloat` results reproduce a 512-bit `BigFloat` reference to the precision dictated by their limb count (≈ 16 correct decimal digits per `Float64` limb).

1 Background: where precision matters in MST

The MST method writes the “in” and “up” radial Teukolsky solutions as series of hypergeometric functions indexed by the *renormalized angular momentum* ν , with expansion coefficients f_n^ν obeying a three-term recurrence [1]. Our solver obtains ν from the monodromy of the confluent Heun equation: it evaluates $\cos(2\pi\nu)$ as a truncated double series and inverts the appropriate branch,

$$\cos(2\pi\nu) = \cos(\pi(\mu_1 - \mu_2)) + \frac{2\pi^2}{a_{1,\Sigma} a_{2,\Sigma}} (-1)^{n-1} a_1^{(n+1)} a_2^{(n+1)}, \quad (1)$$

where the $a_i^{(k)}$ and the closing sums $a_{i,\Sigma}$ are built from a truncation-independent recurrence of length n . The asymptotic amplitudes then follow from sums such as

$$A_+^\nu \propto \frac{\Gamma(\nu + 1 - s + i\epsilon)}{\Gamma(\nu + 1 + s - i\epsilon)} \sum_n f_n^\nu, \quad B^{\text{inc}} = \omega^{-1} (K_\nu - i e^{-i\pi\nu} \sigma K_{-\nu-1}) A_+^\nu e^{i\Theta}, \quad (2)$$

with the matching coefficients K_ν involving ratios of Γ functions. The incidence amplitude B^{inc} controls the Green’s function $G = R_{\text{in}}/(2i\omega B^{\text{inc}})$; cancellations in the branch-cut contribution and in subdominant flux multipoles can lose 20–60 digits, so a working precision well beyond 53 bits is essential there.

2 Three backends, one switch

The numerical core (Eqs. (1)–(2)) is written generically in a real type `R<AbstractFloat` and its complexification `Complex{R}`; the precision is therefore fixed entirely by the type of the inputs (a, ω) . We expose this through a `backend` keyword on the public entry points `compute_nu` and `compute_amplitudes` (and its `nufixed/mero` variants):

backend	working type	precision control
<code>:float64</code>	<code>Float64</code>	fixed, 53 bits
<code>:bigfloat</code>	<code>BigFloat</code>	precision bits (arbitrary)
<code>:multifloat</code>	<code>Float64xN</code> , $N \leq 4$	precision bits $\rightarrow N = \lceil \text{prec}/53 \rceil$ limbs

A single dispatch helper converts the inputs and runs the otherwise-unchanged generic path:

```
nu, p = compute_nu(s, l, m, a, omega; backend=:multifloat, precision=212)
amps = compute_amplitudes(s, l, m, a, omega; backend=:multifloat,
    precision=212)
```

The default (`:auto` on the amplitude routines, `:bigfloat` on `compute_nu`) reproduces the previous behaviour byte-for-byte, so the change is fully backward compatible.

3 What MultiFloat is, and the two caveats it forces

A `MultiFloat{Float64,N}` (alias `Float64xN`) represents a number as an unevaluated sum of N nonoverlapping `Float64` values and implements $+$, $-$, $\times$, $\div$, $\sqrt{}$ with error-free transforms, delivering $\approx 53N$ bits of *mantissa* at a few times the cost of hardware double precision—typically far faster than `BigFloat` for arithmetic-dominated kernels [2]. Two properties of this design shape the integration:

(C1) Mantissa, not exponent. A `MultiFloat` extends the significand but keeps `Float64`’s exponent range ($\sim 10^{\pm 308}$). The monodromy series (1) accumulates factorial/Pochhammer products whose magnitude overflows to `Inf/NaN` at series length $n \gtrsim 170$ —exactly the reason the `Float64` path is capped at $n = 60$. At extended precision the required length grows as $n \approx 4.7 \times (\text{bits})$, far past the overflow point. We therefore evaluate the *scalar* $\cos(2\pi\nu)$ of (1) in `BigFloat` at the `MultiFloat` working precision and convert the single result back; the arithmetic-heavy remainder (the f_n'' recurrence and the amplitude/ K_ν sums, whose terms stay within the exponent range) runs natively in `MultiFloat`.

(C2) Missing transcendentals / Γ . `MultiFloats.jl` does not implement the elementary transcendentals or the Γ function. A compatibility shim (`src/multifloat_compat.jl`, mirroring the existing `arb_compat.jl`) closes the gaps used by MST:

- it enables `MultiFloats`’ sanctioned `BigFloat` fallback for the one-argument transcendentals (`sin`, `cos`, `exp`, `log`, $\dots$);
- it adds the *two*-argument `atan(y, x)` that the fallback omits but on which `Complex log / arccos / arccosh` depend;
- it adds Γ on `Complex{MultiFloat}` (routed through `BigFloat` at the working precision), as in Eq. (2).

These functions appear only $\mathcal{O}(1)$ times per call (prefactors and branch selection), so the `BigFloat` detour is negligible while the dominant arithmetic stays native.

Width limit. The shipped `MultiFloats.jl` provides multiplication/division/square-root kernels only for $N \leq 4$ (`Float64x1...Float64x4`), so the MultiFloat backend tops out at ≈ 212 bits (≈ 63 decimal digits); request more digits via the `:bigfloat` backend. The precision $\rightarrow$ width map clamps accordingly.

4 Validation

Figure 1 shows the relative error of ν and B^{inc} for a representative Kerr mode against a 1024-bit `BigFloat` reference, as a function of working precision. The MultiFloat points (blue) lie on top of the `BigFloat` curve (gray) at every matching bit count, each additional limb buying ≈ 16 decimal digits, until the $N = 4$ ceiling. `Float64` (red) sits at the $\sim 10^{-11}$ level set by the conditioning of this mode.

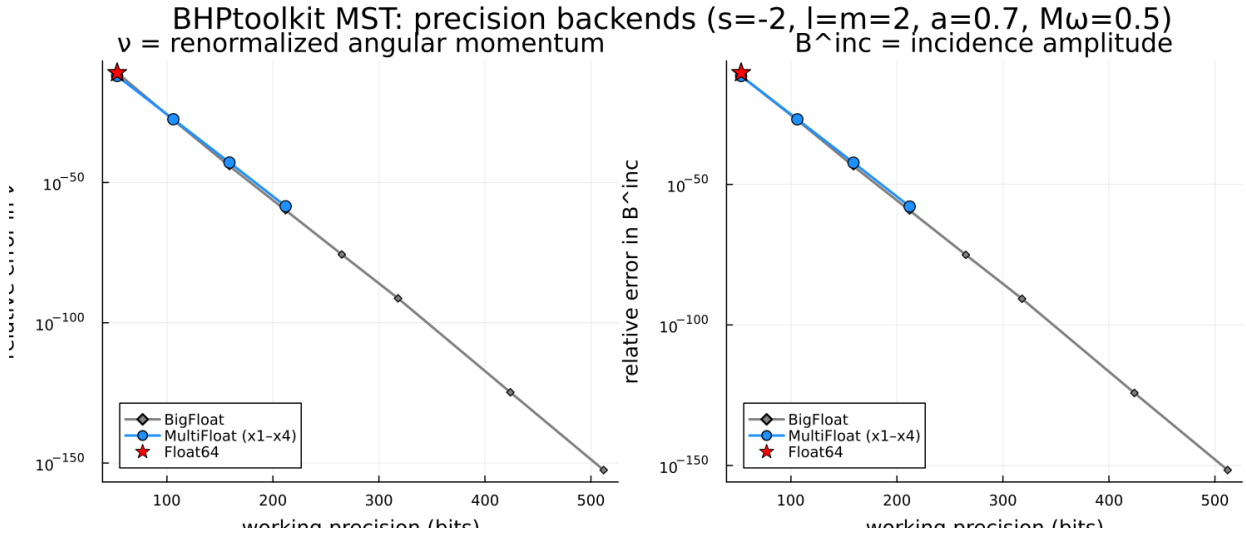


Figure 1: Relative error of the renormalized angular momentum ν (left) and the incidence amplitude B^{inc} (right) versus working precision, for $s = -2$, $\ell = m = 2$, $a = 0.7$, $M\omega = 0.5$, against a 1024-bit `BigFloat` reference. MultiFloat (blue, $N = 1 \dots 4$) matches `BigFloat` (gray) at equal bit width; `Float64` (red) is shown for scale.

backend / width	bits	rel. error in ν
Float64	53	1.3×10^{-11}
Float64x2	106	3.6×10^{-28}
Float64x3	159	1.3×10^{-43}
Float64x4	212	3.4×10^{-59}
BigFloat (212)	212	1.3×10^{-60}

Table 1: Accuracy scaling for the mode of Fig. 1: each MultiFloat limb adds ≈ 16 correct decimal digits, matching `BigFloat` at the same bit width.

A broader sweep of ~ 200 modes (spins $a \in \{0, 0.5, 0.9, 0.99\}$, several (ℓ, m) , and real and complex ω) confirms: every MultiFloat result is finite, beats Float64, and reproduces the BigFloat reference to the expected precision. The dedicated test set lives in `test/test_multifloat_backend.jl`.

5 Summary

The MST solver now offers a uniform three-way precision option. Float64 is the fast default; BigFloat provides unbounded precision for the most demanding branch-cut/flux computations; and MultiFloat occupies the useful middle ground of 30–60 extra-precise digits with native, non-allocating arithmetic, while delegating only its missing transcendentals and the overflow-prone monodromy scalar to BigFloat.

References

- [1] M. Sasaki and H. Tagoshi, *Analytic Black Hole Perturbation Approach to Gravitational Radiation*, Living Rev. Rel. **6** (2003) 6.
- [2] D. K. Zhang, MultiFloats.jl, <https://github.com/dzhang314/MultiFloats.jl>.